

第四天 陣列、字串

大里 APCS 營隊 · 一次裝起一大堆資料

邱皇愷

今天的目標

- 學會**一維陣列**：宣告、初始化、走訪
- 用陣列找**最大值 / 最小值**、計算**平均**
- 認識**二維陣列**：矩陣加總、九宮格
- 處理**字串**：長度、字元走訪、子字串與迴文
- 每教完一段，馬上到 itouOJ 練一題，完成 Day4 的 5 題

開場暖身

回顧昨天

- 迴圈三元素：初始值、條件、更新
- 累加從 0、累乘從 1，變數放在迴圈外面
- 巢狀迴圈執行次數是外層 $\times$ 內層
- 今天要學的，是讓一個名字能裝一整排資料

暖身問題（不用寫程式）

如果要記錄全班 30 個人的分數，用昨天學的方式，你會需要宣告幾個變數？這樣做有什麼不方便？

想 30 秒，等一下公佈參考答案。

暖身問題：參考答案

- 照昨天的做法，要宣告 30 個變數：``s1, s2, s3, ... s30``
- 不方便的地方：要找最大值得寫 30 個 ``if``；要全部印出來得寫 30 行 ``cout``
- 更慘的是，昨天教的迴圈沒辦法套用在這 30 個各自獨立的變數上
- 需要一種方式，讓「一整排資料」可以用一個名字 + 編號存取

這就是**陣列**——今天的主角。有了陣列，昨天學的迴圈就能拿來「一次處理一整排資料」，兩個技巧合體威力大增。

今天會不斷用到「昨天 + 前天」的技巧

Day2

if / else 做決定

Day3

迴圈重複做事

Day4 (今天)

陣列裝一整排資料

三個技巧疊在一起：用迴圈走訪陣列，過程中用 **if** 做判斷，這正是今天所有題目共同的模式。

一維陣列

為什麼需要陣列？

要存 5 個學生的分數，難道要宣告 5 個變數？

```
int s0, s1, s2, s3, s4;    // 😓 那 1000 個學生呢？
```

用陣列：

```
int score[5];              // 一次宣告 5 個整數的格子
```



索引 (index) 從 **0** 開始，`int score[5]` 的最後一格是 `score[4]`。

為什麼索引從 0 開始

- 陣列在記憶體裡是**連續**的一塊空間
- 索引其實代表「從第一格開始，往後跳幾格」
- 第一格本身不用跳，所以是 `[0]`；跳一格是 `[1]`
- 這是 C++（以及大多數程式語言）的慣例，習慣了就不會覺得奇怪

宣告與初始化

```
int a[5] = {90, 85, 78, 92, 88}; // 宣告同時給值
int b[5] = {};                  // 全部初始化為 0

cout << a[0];    // 90
cout << a[4];    // 88

a[2] = 100;      // 修改
```

存取和修改陣列元素的方式完全一樣：`陣列名[索引]`。

存取超出範圍會怎樣

```
int a[5] = {90, 85, 78, 92, 88};  
cout << a[5];    // ⚠ 沒有這一格！  
cout << a[-1];   // ⚠ 也沒有這一格！
```

「`a[5]`」、`a[-1]` 是**未定義行為**——不會有清楚的錯誤訊息，只會讀到隔壁記憶體垃圾資料，或直接讓程式當掉（RE）。這是陣列最危險的地方：**編譯器不會幫你檢查索引對不對。**

陣列開多大：寧可多開，不要剛好

```
int n;           // 題目說  $n \leq 1000$   
cin >> n;  
int a[1005];     // ← 開得比上限大一點
```

多開幾格幾乎不花任何成本，但少開一格就會踩到未定義行為。養成習慣：看到題目的上限，開的陣列大小要比它更大一點。

用迴圈走訪陣列

```
int n;  
cin >> n;  
int a[1005];  
  
for (int i = 0; i < n; i++) {    // 讀進來  
    cin >> a[i];  
}  
for (int i = 0; i < n; i++) {    // 走一遍  
    cout << a[i] << " ";  
}
```

昨天學的 `for` 迴圈，配上陣列的索引，就能一次處理一整排資料。迴圈變數 `i` 剛好可以當成陣列的索引來用。

小測驗：迴圈條件對不對？

```
int n = 5;
int a[10];
for (int i = 0; i <= n; i++) {
    cin >> a[i];
}
```

這段程式有什麼問題？

差一錯誤：條件是 `i <= n`，會跑 `i = 0` 到 `i = 5`，共讀入 **6** 個值，但 `n` 只有 5 個。應該用 `i < n`（或 `i <= n-1`）才對。

找最大值與最小值

```
int mx = a[0], mn = a[0];           // 先假設第 0 個既是最大也是最小
for (int i = 1; i < n; i++) {
    if (a[i] > mx) mx = a[i];        // 比目前的大就更新
    if (a[i] < mn) mn = a[i];        // 比目前的小就更新
}
```

還記得第二天的「三數取最大」嗎？同樣的招式，換成迴圈跑陣列。

起始值陷阱：為什麼不能用 0

```
int mx = 0; // ❌ 危險！  
for (int i = 0; i < n; i++) {  
    if (a[i] > mx) mx = a[i];  
}
```

如果整個陣列都是**負數**（例如 `-5, -3, -8`），沒有一個數字比 0 大，`mx` 會一直卡在 0，變成錯誤答案。**起始值一定要設成 `a[0]`**，用「陣列裡真實存在的值」開始比較才安全。

計算總和與平均

```
long long sum = 0;
for (int i = 0; i < n; i++) {
    sum += a[i];
}
cout << sum << endl;
cout << (double)sum / n << endl;    // 平均要轉型
```

累加的招式跟昨天完全一樣，只是把「數字」換成「陣列裡的每一格」。求平均別忘了第二天教的整數除法陷阱。

為什麼總和要用 ``long long``

- ``int`` 的範圍大約是 ± 21 億
- 如果陣列有 1000 個數字，每個都接近 int 上限，加起來很容易超出 **int 範圍**
- 這種情況叫**溢位 (overflow)**，算出來的總和會變成錯誤的數字，而且不會報錯
- 遇到「很多數字加總」的題目，養成習慣改用 ``long long`` (範圍大很多)

小測驗：這樣寫對嗎？

```
int a[5] = {3, -7, 2, -9, 5};
int mx = 0;
for (int i = 0; i < 5; i++) {
    if (a[i] > mx) mx = a[i];
}
cout << mx;
```

不對。這裡最大值應該是 `5`，但因為起始值設成 `0`，而 `5 > 0` 成立，剛好矇對了。如果最大值是負的（例如全部都是負數），就會錯誤地印出 `0`。

練習：追蹤找最大值

```
int a[4] = {5, 2, 9, 3};
int mx = a[0];
for (int i = 1; i < 4; i++) {
    if (a[i] > mx) mx = a[i];
}
```

自己列表格，追蹤每一輪 `mx` 的變化。

i	ai	ai > mx ?	mx
開始	—	—	5
1	2	否	5
2	9	是	9
3	3	否	9

最終 `mx = 9` 。

陣列常見錯誤總複習

錯誤	長什麼樣子	怎麼避免
索引差一錯誤	<code>`i <= n`</code> 卻該用 <code>`i < n`</code>	想清楚是「幾個」還是「到第幾個」
陣列開太小	<code>`int a[n]`</code> 剛好卡在上限	開得比題目上限大一點
起始值設 0	找最大值卻全是負數	起始值用 <code>`a[0]`</code> ，不要用 0
溢位	總和超過 int 範圍	總和用 <code>`long long`</code>

練習時間

第 12 題 最大最小平均

輸入 n 個整數，輸出最大值、最小值、平均值（四捨五入至小數點後 **2** 位）。

輸入

5

3 7 2 9 4

輸出

9

2

5.00

第 12 題 讀題

- 輸入是 n ，再來一整排 n 個整數
- 輸出三行：最大值、最小值、平均值
- 平均值要**固定兩位小數**，就算剛好整除也要印 ``.00``

第 12 題 規劃骨架

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;
    int a[1005];
    for (int i = 0; i < n; i++) cin >> a[i];
    // 這裡找最大最小、算平均
    return 0;
}
```

第 12 題 邊走訪邊記錄

```
int mx = a[0], mn = a[0];
long long sum = 0;
for (int i = 0; i < n; i++) {
    if (a[i] > mx) mx = a[i];
    if (a[i] < mn) mn = a[i];
    sum += a[i];
}
```

一個迴圈裡同時做三件事：比大小找最大、比大小找最小、順便累加。不需要為每件事各寫一個迴圈。

第 12 題 輸出固定兩位小數

```
#include <cstdio>    // ← printf 需要這個
// ...
cout << mx << endl << mn << endl;
printf("%.2f\n", (double)sum / n);
```

⚠ 平均要**固定兩位小數**：5 要輸出 `5.00`，不是 `5`。 `printf("%.2f")` 是最簡單的做法，記得先 `#include <cstdio>`。

第 12 題 完整程式

```
#include <iostream>
#include <cstdio>
using namespace std;

int main() {
    int n;
    cin >> n;
    int a[1005];
    for (int i = 0; i < n; i++) cin >> a[i];

    int mx = a[0], mn = a[0];
    long long sum = 0;
    for (int i = 0; i < n; i++) {
        if (a[i] > mx) mx = a[i];
        if (a[i] < mn) mn = a[i];
        sum += a[i];
    }
    cout << mx << endl << mn << endl;
    printf("%.2f\n", (double)sum / n);
    return 0;
}
```

動手做

打開 oj.itousouta.me → 課程 Day4 → 第 12 題

寫到 AC 為止，再往下聽

二維陣列基礎

二維陣列：像表格一樣

```
int m[3][3] = {  
    {1, 2, 3},  
    {4, 5, 6},  
    {7, 8, 9}  
};
```

	j=0	j=1	j=2
i=0	1	2	3
i=1	4	5	6
i=2	7	8	9

`m[1][2]` = **6**。第一個 `i` 是「第幾列」，第二個 `j` 是「第幾行」。

二維陣列其實是「陣列的陣列」

- 一維陣列是一排格子；二維陣列是一排「一排格子」
- `m[3][3]` 可以想成：先有 3 個一維陣列，每個裡面又有 3 格
- 存取時要給**兩個**索引：先選第幾排（列），再選那一排裡的第幾格（行）

走訪二維陣列：巢狀迴圈

```
int n, m;  
cin >> n >> m;  
int a[105][105];  
  
for (int i = 0; i < n; i++) {           // 每一列  
    for (int j = 0; j < m; j++) {       // 每一行  
        cin >> a[i][j];  
    }  
}
```

昨天學的巢狀迴圈，在這裡正好派上用場。讀入、走訪、加總——二維陣列的一切都是這個雙層 for。

二維陣列開多大

```
int a[105][105];    // 如果  $n, m \leq 100$ ，兩維都要開大一點
```

二維陣列要**兩個維度都**開得比上限大，只開對一邊沒有用。題目說 $n, m \leq 100$ ，兩維都至少要開 101 以上。

練習：追蹤二維陣列走訪

```
int m[2][2] = {{1, 2}, {3, 4}};  
for (int i = 0; i < 2; i++) {  
    for (int j = 0; j < 2; j++) {  
        cout << m[i][j] << " ";  
    }  
}
```

會依序印出哪些數字？

1 2 3 4 ——外層 `i=0` 時，內層 `j` 跑 0,1，印出 `m[0][0]=1`、`m[0][1]=2`；外層 `i=1` 時同樣跑一輪，印出 `m[1][0]=3`、`m[1][1]=4`。

小測驗：這是第幾列第幾行？

```
int m[4][4];  
// ...  
cout << m[2][0];
```

`m[2][0]` 存取的是第幾列、第幾行（從 0 算起）？

****第 2 列、第 0 行**。**第一個索引永遠是列，第二個永遠是行。

練習時間

第 13 題 矩陣加總

輸入 $n \times m$ 的矩陣，輸出所有元素總和。

輸入

```
2 3
1 2 3
4 5 6
```

輸出

```
21
```

第 13 題 想清楚：需要真的開陣列嗎？

- 題目只要「總和」，不需要之後再用到矩陣裡個別的值
- 既然如此，**邊讀邊加**就好，不必開一個二維陣列存起來
- 這是很重要的判斷力：先想清楚題目要什麼，再決定要不要開資料結構

第 13 題 程式

```
int n, m;
cin >> n >> m;

long long sum = 0;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        int x;
        cin >> x;
        sum += x;          // ← 讀進來馬上加，不用存
    }
}
cout << sum << endl;
```

會用二維陣列不代表每次都要用；先想清楚題目要什麼。

第 13 題 如果真的開了陣列也不算錯

```
int a[105][105];
for (int i = 0; i < n; i++)
    for (int j = 0; j < m; j++) cin >> a[i][j];

long long sum = 0;
for (int i = 0; i < n; i++)
    for (int j = 0; j < m; j++) sum += a[i][j];
```

這樣寫**也會 AC**，只是多寫了一個用不到的陣列，多跑一次迴圈。兩種寫法都對，差別只在效率和簡潔度——先求對，再求好。

動手做

打開 oj.itousouta.me → 課程 Day4 → 第 13 題

寫到 AC 為止，再往下聽

二維陣列進階

同一個迴圈，同時走列與行

```
for (int i = 0; i < 3; i++) {  
    int row = 0, col = 0;  
    for (int j = 0; j < 3; j++) {  
        row += a[i][j];    // 第 i 列  
        col += a[j][i];    // 第 i 行  
    }  
}
```

「`a[i][j]`」和「`a[j][i]`」只差在索引順序——一個走列、一個走行，可以用**同一個迴圈**同時處理，不用寫成兩段。之後檢查九宮格會直接用到這個技巧。

練習時間

第 14 題 九宮格驗證 (幻方)

輸入 3×3 方格，判斷每列、每行、兩條對角線的總和是否全部相等。

4	9	2
3	5	7
8	1	6

每列、每行、對角線都 = 15

要檢查 **8** 條線：

- 3 條列
- 3 條行
- 2 條對角線

第 14 題 讀題：這題要檢查什麼

- 3 條列：每一列三個數字加起來
- 3 條行：每一行三個數字加起來
- 1 條主對角線：左上到右下
- 1 條反對角線：右上到左下
- 全部 8 條線的總和都要等於同一個數，才輸出 ``Magic``

第 14 題 為什麼不能只檢查列？

如果只檢查 3 條列的總和相等，會不會誤判？

會。每列總和相等，不代表行和對角線也相等——例如把某一系列的數字順序打亂，列總和不變，但行的總和就會跟著錯。**8 條線都要檢查，缺一不可。**

第 14 題 定基準：以第一列為標準

```
int a[3][3];
for (int i = 0; i < 3; i++)
    for (int j = 0; j < 3; j++) cin >> a[i][j];

int target = a[0][0] + a[0][1] + a[0][2];    // 以第一列為基準
bool ok = true;
```

先算出第一列的總和當作「標準答案」，之後每一條線都跟它比對，只要有一條不一樣，就代表不是幻方。

第 14 題 同時檢查列與行

```
for (int i = 0; i < 3; i++) {  
    int row = 0, col = 0;  
    for (int j = 0; j < 3; j++) {  
        row += a[i][j];    // 第 i 列  
        col += a[j][i];    // 第 i 行  
    }  
    if (row != target || col != target) ok = false;  
}
```

第 14 題 檢查兩條對角線

```
if (a[0][0] + a[1][1] + a[2][2] != target) ok = false; // 主對角線
if (a[0][2] + a[1][1] + a[2][0] != target) ok = false; // 反對角線

cout << (ok ? "Magic" : "Not Magic") << endl;
```

對角線只有兩條，數量固定，直接手動列出來比迴圈更簡單清楚。

第 14 題 完整程式

```
int a[3][3];
for (int i = 0; i < 3; i++)
    for (int j = 0; j < 3; j++) cin >> a[i][j];

int target = a[0][0] + a[0][1] + a[0][2];
bool ok = true;

for (int i = 0; i < 3; i++) {
    int row = 0, col = 0;
    for (int j = 0; j < 3; j++) { row += a[i][j]; col += a[j][i]; }
    if (row != target || col != target) ok = false;
}
if (a[0][0] + a[1][1] + a[2][2] != target) ok = false;
if (a[0][2] + a[1][1] + a[2][0] != target) ok = false;

cout << (ok ? "Magic" : "Not Magic") << endl;
```

動手做

打開 oj.itousouta.me → 課程 Day4 → 第 14 題

寫到 AC 為止，再往下聽



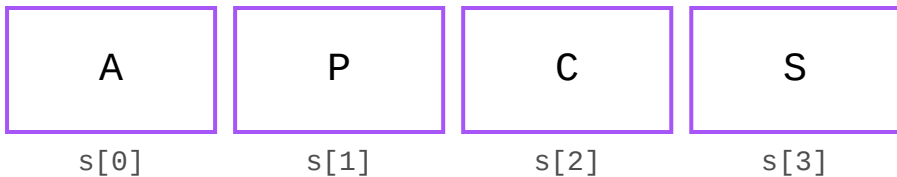
中場休息

15 分鐘後回來，我們進入「字串」

字串基礎

字串就是一串字元

```
string s = "APCS";
```



```
cout << s.size();    // 4    (長度)  
cout << s[0];        // A    (像陣列一樣用索引取字元)
```

「string」其實就是「一排 char」，走訪方式跟陣列一模一樣。

讀入字串

```
string s;  
cin >> s;           // 讀一個「單字」（遇到空白就停）
```

「cin >>」讀字串時，遇到空白或換行就會停止。如果字串裡有空格（例如一整句話），要改用「getline(cin, s)」讀一整行。今天的題目都是單字，用「cin >>」就好。

``cin >>`` vs ``getline`` : 差在哪

```
`cin >> s`
```

只讀到第一個空白為止。

輸入 ``hello world`` 只會讀到 ``hello``

```
`getline(cin, s)`
```

讀一整行，包含中間的空白。

輸入 ``hello world`` 會完整讀到

今天的 5 題輸入都不含空白，用 ``cin >>`` 就足夠。之後遇到「一整句話」的輸入格式時，記得改用 ``getline``。

走訪字串的每個字元

```
for (int i = 0; i < s.size(); i++) {  
    cout << s[i] << " ";  
}  
  
for (char c : s) {           // 比較簡潔的寫法  
    cout << c << " ";  
}
```

兩種寫法效果一樣。第二種叫「range-based for」，讀作「對 s 裡的每個字元 c」，不用自己管索引，看到就直接記住這個寫法即可。

字串比對與串接

```
string a = "abc", b = "abc", c = "xyz";  
  
cout << (a == b);           // 1    直接用 == 比較  
string full = a + c;        // abcxyz
```

C++ 的 `string` 可以直接用 `==` 比較、用 `+` 串接，比 C 語言方便很多。

`substr` : 取一段字串

```
string s = "programming";  
cout << s.substr(0, 4);      // prog   從索引 0 取 4 個字元  
cout << s.substr(4, 3);      // ram     從索引 4 取 3 個字元  
cout << s.substr(7);         // ming    從索引 7 取到最後
```

`substr(起點, 長度)` —— 長度可以省略，代表取到字串結尾。等一下子字串搜尋那題會大量用到。

應用：統計字元

計算字串裡有幾個母音 (a, e, i, o, u) 。

```
string s;  
cin >> s;  
int count = 0;  
  
for (char c : s) {  
    if (c=='a' || c=='e' || c=='i' || c=='o' || c=='u') {  
        count++;  
    }  
}  
  
cout << count;
```

字元也可以拿來比較大小

```
char c = 'd';  
cout << (c >= 'a' && c <= 'z');    // 1, 判斷是不是小寫字母
```

字元在電腦裡本質上是數字，所以可以直接用 `>=`、`<=` 比較，`'a'` 到 `'z'` 剛好是連續的一段數字，這個技巧之後判斷英文字母時很好用。

小測驗：這樣走訪對嗎？

```
string s = "hi";  
for (int i = 0; i <= s.size(); i++) {  
    cout << s[i];  
}
```

不對，又是差一錯誤。`s.size()` 是 2，條件 `i <= 2` 會讓 `i` 跑到 2，但合法索引只有 `s[0]`、`s[1]`，`s[2]` 已經超出範圍。應該用 `i < s.size()`。

小測驗：substr 用法

```
string s = "itousouta";  
cout << s.substr(4, 4);
```

****`sout`****——從索引 4（第 5 個字元 `s`）開始，取 4 個字元。

練習時間

第 15 題 子字串搜尋

輸入字串 s 與 t ，判斷 t 是否為 s 的連續一段。

p r o g r a m m i n g 找 gram → Yes

第 15 題 想法：每個起點都試一次

- 從 s 的每一個位置開始，切出跟 t 一樣長的一段
- 拿這一段跟 t 比對，一樣就是找到了
- 起點不能太後面——剩下的長度要至少跟 t 一樣長才夠切

第 15 題 用範例走一次

`s = "programming"` , `t = "gram"` (長度 4)

起點 i	s.substr(i, 4)	等於 t ?
0	<code>`prog`</code>	否
1	<code>`rogr`</code>	否
2	<code>`ogra`</code>	否
3	<code>`gram`</code>	 是 !

找到後立刻 ``break`` , 不用把剩下的起點都試完。

第 15 題 程式

```
string s, t;
cin >> s >> t;

bool found = false;
for (int i = 0; i + t.size() <= s.size(); i++) {
    if (s.substr(i, t.size()) == t) { found = true; break; }
}
cout << (found ? "Yes" : "No") << endl;
```

迴圈條件是 `i + t.size() <= s.size()` ——起點不能太後面，否則剩下的長度不夠比。找到就用 `break` 提早結束，不用試完所有起點。

第 15 題 也可以這樣寫

```
cout << (s.find(t) != string::npos ? "Yes" : "No") << endl;
```

`s.find(t)` 是 `string` 內建的搜尋功能，找不到會回傳特殊值 `string::npos`。兩種寫法效果一樣，今天先知道自己動手寫的版本在做什麼，之後熟悉了可以改用這個更精簡的寫法。

第 15 題 常見錯誤：迴圈條件寫錯

```
for (int i = 0; i < s.size(); i++) {           // ✗ 少考慮 t 的長度
    if (s.substr(i, t.size()) == t) { ... }
}
```

如果 `i` 太靠近 `s` 的尾端，`s.substr(i, t.size())` 會取不滿 `t` 的長度（字串長度不夠時 `substr` 只會回傳剩下的部分），導致比對永遠不相等，或在某些情況下引發例外。條件一定要包含 `i + t.size() <= s.size()` 的檢查。

動手做

打開 oj.itousouta.me → 課程 Day4 → 第 15 題

寫到 AC 為止，再往下聽

雙指標技巧

雙指標：從頭尾往中間夾

l e v e l

↑ ↑

頭尾往中間夾

```
int i = 0, j = s.size() - 1;
while (i < j) {
    // 比較 s[i] 跟 s[j]
    i++; j--;
}
```

這個技巧叫**雙指標**：兩個位置同時往中間移動，只要走到一半（`i < j` 不成立）就能檢查完整個字串。等一下迴文判斷就是雙指標最經典的應用。

練習時間

第 16 題 迴文判斷

輸入字串，判斷正著讀和反著讀是否相同。

l e v e l

輸入

level

輸出

Yes

第 16 題 想法：套用雙指標

- 一個指標 `i`` 從最前面開始，一個指標 `j`` 從最後面開始
- 比較 `s[i]`` 跟 `s[j]``，不一樣就不是迴文
- 一樣的話，`i`` 往後移、`j`` 往前移，繼續比對
- 直到 `i`` 跟 `j`` 在中間相遇（或錯過）為止

第 16 題 程式

```
string s;  
cin >> s;  
  
bool ok = true;  
int i = 0, j = s.size() - 1;  
while (i < j) {  
    if (s[i] != s[j]) { ok = false; break; }  
    i++; j--; // 兩邊一起往中間走  
}  
cout << (ok ? "Yes" : "No") << endl;
```

只要走一半就能得到答案，比「反轉整個字串再比對」更快，而且不需要額外開一個字串來存反轉的結果。

第 16 題 逐步追蹤 "level"

步驟	i	j	si	sj	相等？
1	0	4	l	l	
2	1	3	e	e	
3	2	2	—	—	i=j，停止

`i`` 跟 `j`` 相遇時（`i < j`` 不再成立），代表已經檢查完整個字串，中間那個字元不需要跟自己比。

第 16 題 偶數長度也適用嗎？

`s = "abba"` 追蹤一次：

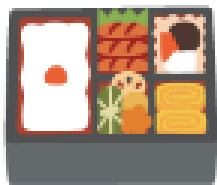
步驟	i	j	si	sj	相等？
1	0	3	a	a	✓
2	1	2	b	b	✓
—	2	1	—	—	i > j，停止

偶數長度時 `i` 會直接跳過 `j`（`i > j`），不會有兩個指標相等的情況，但迴圈條件 `i < j` 一樣正確處理了這種情況，不需要特別分兩種寫法。

動手做

打開 oj.itousouta.me → 課程 Day4 → 第 16 題

寫到 AC 為止——今天 5 題全部完成！



短暫休息

10 分鐘後回來，做綜合小考

隨堂小測驗

Q1

`int a[5];` 合法的索引範圍是多少？

`**a[0]` 到 `a[4]**`，總共 5 格。

Q2

找陣列最大值時，起始值為什麼要設成 `a[0]` 而不是 `0` ？

因為陣列可能**全部是負數**，這種情況下沒有任何值比 0 大，起始值設 0 會讓答案永遠錯誤地停在 0。

Q3

`m[2][3]` 存取的是第幾列、第幾行？

****第 2 列、第 3 行****。第一個索引是列，第二個是行。

Q4

`s.substr(2, 3)` 是從哪裡開始、取幾個字元？

****從索引 2 開始，取 3 個字元**。**

Q5

雙指標判斷迴文時，`i`` 和 `j`` 分別往哪個方向移動？

`i`` 從頭往後 (`i++``)，`j`` 從尾往前 (`j--``)，兩個一起往中間靠攏。

Q6

如果陣列裡的數字總和可能超過 21 億，應該用哪個型態存總和？

「`long long`」——`int` 的範圍大約只有 ± 21 億，超過會溢位算錯。

Q7

``cin >> s`` 讀取字串時，遇到什麼會停止？

****空白或換行****。要讀含空白的一整行要用 ``getline(cin, s)``。

Q8

矩陣加總那一題，為什麼可以「邊讀邊加」而不用真的開二維陣列？

因為題目只要**總和**，不需要之後再用到矩陣裡個別的值，讀進來的當下就能直接累加，不必先存起來。

Q9

`string s = "hello"; cout << s.substr(1);` 會印出什麼？

`**ello**`——`substr` 省略長度時，代表取到字串結尾。

Q10

九宮格驗證那一題，為什麼只檢查 3 條列是不夠的？

列總和相等不代表行、對角線也相等——把某列數字打亂，列總和不變，但其他線可能就不相等了。8 條線缺一不可。

分組討論

討論①

跟旁邊同學互相解釋一次：為什麼找最大值的起始值要用 `a[0]`，不能用 `0`？想一個會讓 `0` 這個起始值出錯的具體例子。

討論②

二維陣列 `m[i][j]` 跟 `m[j][i]`，如果矩陣不是正方形（列數 $\neq$ 行數），這兩個寫法可能會出什麼問題？跟同學討論看看。

討論③

`s[i]` 走訪字串和 `a[i]` 走訪陣列，寫法幾乎一模一樣。跟旁邊同學討論：這兩者本質上為什麼可以用同一種方式處理？

討論④

如果要用陣列存「全班期中考跟期末考」兩次成績，你會怎麼設計資料結構？需要幾維的陣列？跟旁邊同學討論看看。

今日重點回顧

回顧①：一維陣列

- 索引從 **0** 開始，`a[n]` 的最後一格是 `a[n-1]`
- 陣列要開得比題目上限**大一點**，避免踩到未定義行為
- 找最大最小的起始值要用 `a[0]`，不要用 0

回顧②：二維陣列

- `m[i][j]`：`i` 是列、`j` 是行，用巢狀迴圈走訪
- 兩個維度都要開得比上限大
- 只要總和的話，邊讀邊加就好，不必真的開陣列

回顧③：字串

- 字串可用 `s[i]` 取字元、`s.size()` 取長度、`substr` 取一段
- 走訪字串跟走訪陣列的方式一模一樣
- **雙指標**從頭尾往中間夾，適合迴文這類對稱問題

回顧④：今天的節奏

教了什麼

馬上練

一維陣列

第 12 題

二維陣列基礎

第 13 題

二維陣列進階

第 14 題

字串基礎

第 15 題

雙指標技巧

第 16 題

教一段、練一題，跟前兩天一樣。

四天累積下來，你已經會的事

- Day1：輸入、輸出、程式的基本骨架
- Day2：用 if / else 讓程式做決定
- Day3：用迴圈讓程式重複做事
- Day4（今天）：用陣列、字串裝一整排資料

這四樣工具組合起來，已經能解決非常多實際的問題了。

明天預告

明天是總複習與實戰演練，把這四天的功力全部用上！

不會教新語法，而是練習「拿到題目該怎麼想」——這才是寫程式真正的核心能力。

謝謝大家

明天見